

Java control statements

30 September 2026 06:33

Java runtime environment (JVM) executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is the one of the fundamental feature of Java which provides a smooth flow of program.

Types of Control Flow Statements

Java provides three types of control flow statements.

1. Decision making statements.

- i. if statements
- ii. switch statements

2. Loop statements

- i. do...while loop
- ii. while loop
- iii. for loop
- iv. for-each loop

3. Jump statements

- i. break
- ii. continue

Java Decision-Making Statements

As the name suggests, decision - making statements decide which statement to execute and when. Decision - making statements evaluate the boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision making statement in Java. That is 'if' a statement and 'switch' statement.

'if' Statement

Syntax:

```
if(condition){
    //if true execute the block
}
```

'if...else' statement

Syntax:

```
if(condition){
    //if true execute the block
}
else{
    //if false execute the block
}
```

'if...else if' statement

Syntax:

```
if(condition){
    //if true execute the block
}
else if{
    //if false execute the block
}
else if{
    //if false execute the block
}
else if{
    //if false execute the block
}
...
else{
    //if all the 'if' block fails to execute
    //this block will execute, we call it dangling else
}
```

'switch' Statement

In Java, switch statements are similar to if - else - if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if - else - if statement. It also enhances the readability of the program.

Points to be noted about switch statement:

- The case variable can be int, short, byte, char or enumeration. String type is also supported since version 7 of Java.
- Cases cannot be duplicated.
- Default statement is executed when any of the case doesn't match the value of the expression. It is optional.
- Break statement terminates the switch block when the condition is satisfied. It is optional if not used next case is executed.
- While using switch statements, we must notice that the case expression will be the same type as the variable. However, it also be a constant value.

Syntax:

```
switch(expression){
    case value1:
        Statement(s)
        break;
    case value2:
        Statement(s)
        break;
    case value3:
```

```

        Statement(s)
        break;
    case value4:
        Statement(s)
        break;
    ...
    default:
        Statement(s)

```

Data-Input through keyboard

```
Scanner sc = new Scanner(System.in);
```

Scanner class is present in java.util package, so we need to import the java.util package to create the object of Scanner class.

Scanner class contains methods for data input in Java.

Data type	Method from Scanner class
boolean	nextBoolean()
byte	nextByte()
short	nextShort()
int	nextInt()
long	nextLong()
float	nextFloat()
double	nextDouble()
String	next() => it take a word nextLine() => you can input a sentence
char	next().charAt(0)

```
package Chapter2;
import java.util.*;
```

```

/**
 * Write a description of class Q4 here.
 * A shopkeeper offers 80% discount on purchasing articles whereas
 * the other shopkeeper offers two successive discount 60% and 25%
 * for purchasing the same articles.
 * Write a program in Java to compute and display which is better
 * offer for a customer. Take the price of an article as an input.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Q4
{

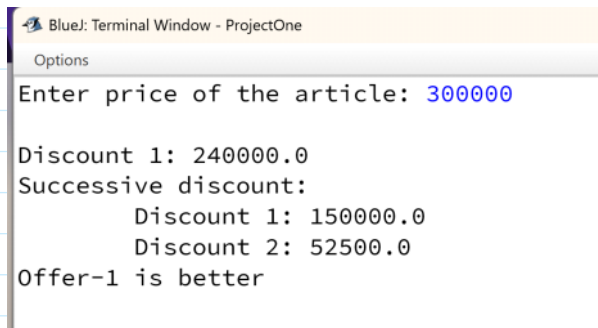
```

```

public static void main(String args[]){
    System.out.print("\f"); //clears terminal window
    Scanner sc = new Scanner(System.in);
    float price, dis1, dis2, dis3;
    System.out.print("Enter price of the article: ");
    price = sc.nextFloat();
    dis1 = price*0.8f;
    dis2 = price*0.5f;
    dis3 = (price-dis2)*0.35f;
    System.out.print("\nDiscount 1: " + dis1);
    System.out.print("\nSuccessive discount: ");
    System.out.print("\n\tDiscount 1: " + dis2);
    System.out.print("\n\tDiscount 2: " + dis3);
    if(dis1 > (dis2+dis3)){
        System.out.print("\nOffer-1 is better");
    }
    else{
        System.out.print("\nOffer-2 is better");
    }
}
}

```

Output:



```

BlueJ: Terminal Window - ProjectOne
Options
Enter price of the article: 300000

Discount 1: 240000.0
Successive discount:
    Discount 1: 150000.0
    Discount 2: 52500.0
Offer-1 is better

```

1. Loop statements

- i. do...while loop
- ii. while loop
- iii. for loop
- iv. for-each loop

2. Jump statements

- i. break
- ii. continue

In Java Loops are divided into 2 categories:

- 1) **Entry control loop:** In entry control loop, we test the condition, if the condition is true, then loop body will execute.

a) while loop

Syntax:

```
while(<condition>){
    //code will execute if condition is true
}
```

b) for loop

Syntax:

```
for(<initialization(s)>; <condition>; <update statement(s)>){
    //if condition is true body will be executed
}
```

c) for-each loop: it only works with collection type(Arrays).

2) **Exit control loop:** In exit control loop, the body of the loop will execute at least once. And, the condition is tested, if true the body of the loop will execute again. Here, condition is tested on exit point of the loop.

a) do...while loop

Syntax:

```
do{
    //code to execute
    ...
}while(<condition>;
```

```
package Looping;
import java.util.*;
```

```
/**
```

```
 * Write a description of class WhileDemo here.
```

```
 * Program to print multiplication table of a number from 1 to N
```

```
 * @author (your name)
```

```
 * @version (a version number or a date)
```

```
 */
```

```
public class WhileDemo
{
```

```
    public static void main(String[] args){
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int num, N;
```

```
        System.out.print("\nEnter number for which multiplication table you
```

```
want: ");
```

```
        num = sc.nextInt();
```

```
        System.out.print("How many times? \n");
```

```
        N = sc.nextInt();
```

```
        int prod, start = 1;
```

```
        while(start <= N){
```

```
            prod = start * num;
```

```
            System.out.print("\n" + start + " * " + num + " = " + prod);
```

```
            start++;
```

```
        }
```

```
    }
```

function/method

No requirement of Object to call this method. It needs class reference.

Server

MCP

client

Model Context Protocol

{ public
private

```

    }
}

```

{ public
private
protected

Output:

```

BlueJ: Terminal Window - ProjectOne
Options
Enter number for which multiplication table you want: 19
How many times?
10
1 * 19 = 19
2 * 19 = 38
3 * 19 = 57
4 * 19 = 76
5 * 19 = 95
6 * 19 = 114
7 * 19 = 133
8 * 19 = 152
9 * 19 = 171
10 * 19 = 190

```

a) for loop

Syntax:

```

for(<initialization(s)>; <condition>; <update statement(s)>){
    //if condition is true body will be executed
}

```

```

package Looping;
import java.util.*;

```

```

/**
 * Write a description of class ForDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */

```

```

public class ForDemo
{

```

```

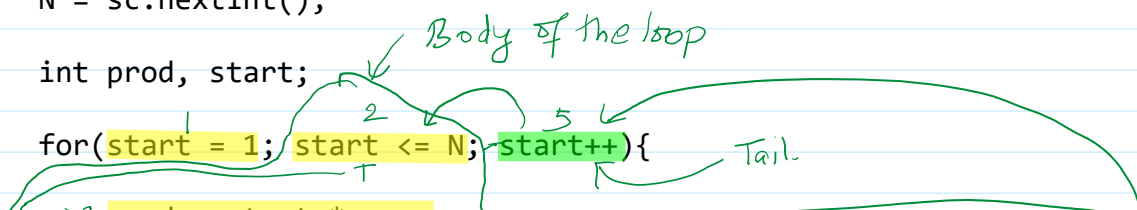
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int num, N;
        System.out.print("\fEnter number for which multiplication table you
want: ");
        num = sc.nextInt();
        System.out.print("How many times? \n");
        N = sc.nextInt();

```

```

        int prod, start;
        for(start = 1; start <= N; start++){

```




```

int prod, start = 1;
do{
    prod = start * num;
    System.out.print("\n" + start + " * " + num + " = " + prod);
    start++;
}while(start <= N);
}
}

```

Testing the condition on exit.

Output:

```

BlueJ: Terminal Window - ProjectOne
Options
Enter number for which multiplication table you want: 37
How many times?
10
1 * 37 = 37
2 * 37 = 74
3 * 37 = 111
4 * 37 = 148
5 * 37 = 185
6 * 37 = 222
7 * 37 = 259
8 * 37 = 296
9 * 37 = 333
10 * 37 = 370

```

```

package Looping;
import java.util.*;

```

```

/**
 * Write a description of class SumOfDigits here.
 * Program asking for a number which should be greater than 0,
 * the program will find sum of its digits.
 * @author (your name)
 * @version (a version number or a date)
 */

```

```

public class SumOfDigits
{

```

```

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        int d, t, num, sumOfDigits = 0;

```

This is validation check.

```

    do{
        System.out.print("Enter any integer greater than 0: ");
        num = sc.nextInt();
        if(num <= 0){
            System.out.print("\nFollow the input instruction.\n");
        }
    }while(num <= 0);
}

```

```
    }  
    }while(num <= 0);
```

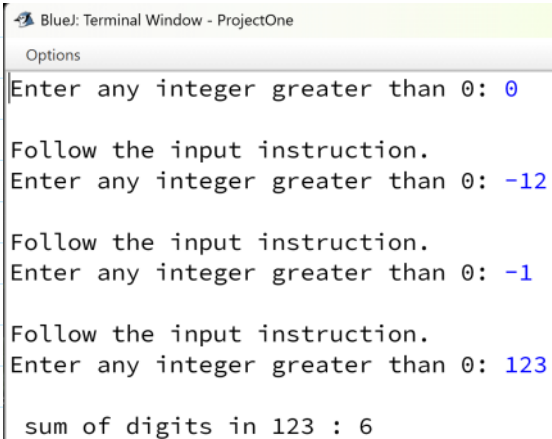
```
    t = num;  
    while(t > 0){  
        d = t % 10;  
        sumOfDigits += d;  
        t /= 10; //remove the last digit  
    }
```

```
    System.out.print("\n sum of digits in " + num + " : " + sumOfDigits);
```

```
    }
```

```
}
```

Output:



```
BlueJ: Terminal Window - ProjectOne  
Options  
Enter any integer greater than 0: 0  
  
Follow the input instruction.  
Enter any integer greater than 0: -12  
  
Follow the input instruction.  
Enter any integer greater than 0: -1  
  
Follow the input instruction.  
Enter any integer greater than 0: 123  
  
sum of digits in 123 : 6
```